A Neural Network with One Hidden Layer, from Scratch

Lecture 7: Neural networks, Parts 7a to 7c

Khiem Nguyen

28 October 2026

James Watt School of Engineering, University of Glasgow

Table of contents

1 Objectives 1

2 Tools 2

3 The data: a flower 2
- 3.1 A baseline: logistic regression . . . 3

4 The network 4
- 4.1 Activation functions . . . 4
- 4.2 Initialisation . . . 5
- 4.3 Forward propagation . . . 5
- 4.4 The cost . . . 6

5 Backpropagation 6
- 5.1 Check 1: the lecture's worked example . . . 7
- 5.2 Check 2: finite differences . . . 7

6 Training with gradient descent 8

7 Why must the hidden activation be nonlinear? 9

8 Challenges 10

9 Summary 10

1 Objectives

In this tutorial you will build a neural network with one hidden layer in NumPy, without any deep-learning library. You will

- implement forward propagation for a whole batch of examples, stored as rows as in the lecture and in PyTorch;
- implement backpropagation, and check it on the lecture's worked example and with finite differences;

- train the network with gradient descent on data that logistic regression cannot separate;
- see why the activation function of the hidden layer must be nonlinear.

2 Tools

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

np.set_printoptions(precision=4, suppress=True)   # print arrays with 4 decimals,
 ↪  no e-notation
```

3 The data: a flower

We generate 400 points in the plane that form a "flower" with two classes, $y = 0$ and $y = 1$. Each class makes up alternate petals, so no straight line can separate them.

```
def load_flower(m=400, seed=42):
    """Two-class 'flower' data: X of shape (m, 2), labels y of shape (m,)."""
    rng = np.random.default_rng(seed)
    X, y = np.zeros((m, 2)), np.zeros(m, dtype=int)
    n = m // 2                                              # points per class
    for j in range(2):
        idx = slice(n * j, n * (j + 1))
        theta = np.linspace(3.12 * j, 3.12 * (j + 1), n) + rng.normal(0, 0.2, n)
 ↪  # angle
        r = 4 * np.sin(4 * theta) + rng.normal(0, 0.2, n)
 ↪  # radius
        X[idx] = np.column_stack([r * np.sin(theta), r * np.cos(theta)])
        y[idx] = j
    return X, y

X, y = load_flower()
print(f"X.shape = {X.shape}, y.shape = {y.shape}")

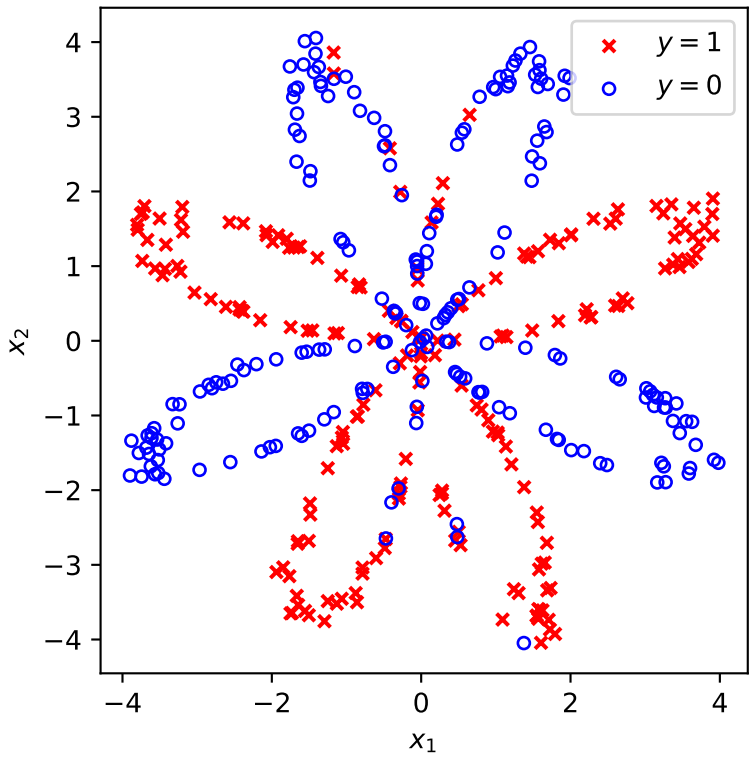
def plot_data(X, y, ax=None):
    """Two-feature data: red crosses for y = 1, blue circles for y = 0."""
    ax = plt.gca() if ax is None else ax
    ax.scatter(X[y == 1, 0], X[y == 1, 1], marker='x', c='r', s=20, label='$y =
 ↪  1$')
    ax.scatter(X[y == 0, 0], X[y == 0, 1], marker='o', facecolor='none',
 ↪  edgecolor='b', s=20,
               label='$y = 0$')
    ax.set(xlabel='$x_1$', ylabel='$x_2$', aspect='equal')

def plot_decision_regions(predict_fn, X, y, ax=None, title=None):
    """Colour the plane by the predicted class (blue: 0, red: 1) and plot the
     ↪  data on top."""
    ax = plt.gca() if ax is None else ax
    g1, g2 = np.meshgrid(np.linspace(-4.5, 4.5, 300), np.linspace(-4.5, 4.5,
 ↪  300))
    y_hat = predict_fn(np.column_stack([g1.ravel(),
 ↪  g2.ravel()])).reshape(g1.shape)
```

```
    ax.contourf(g1, g2, y_hat, levels=[-0.5, 0.5, 1.5], colors=['tab:blue',
↪   'tab:red'], alpha=0.15)
    plot_data(X, y, ax)
    if title:
        ax.set_title(title)

plt.figure(figsize=(4.5, 4.5))
plot_data(X, y)
plt.legend(loc='upper right')
plt.show()
```

```
X.shape = (400, 2), y.shape = (400,)
```



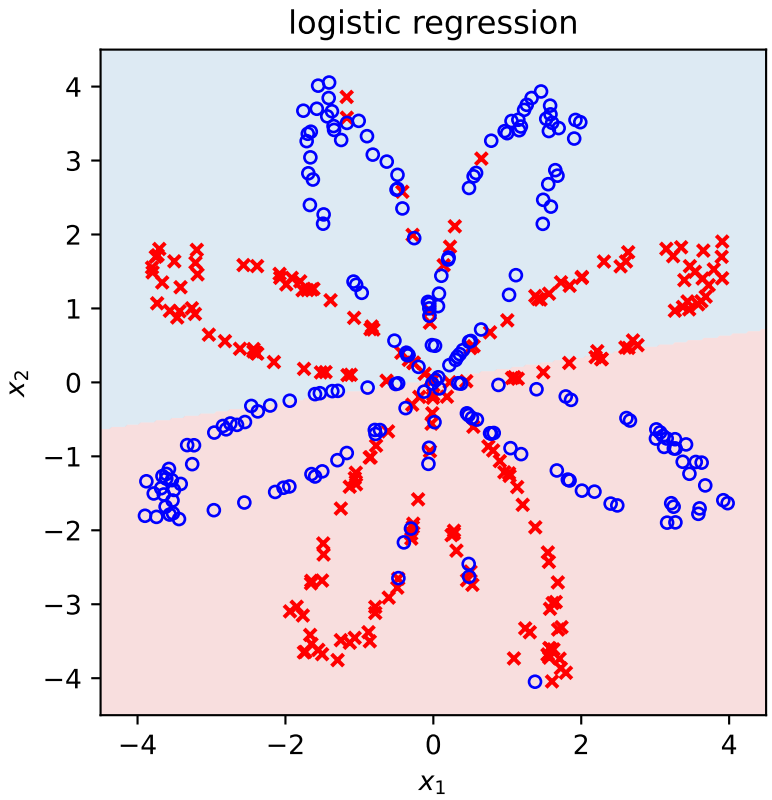
3.1 A baseline: logistic regression

Logistic regression (Lecture 3) has a straight decision boundary, so it cannot follow the petals:

```
log_reg = LogisticRegression().fit(X, y)
print(f"logistic regression, training accuracy: {log_reg.score(X, y):.3f}")

plt.figure(figsize=(4.5, 4.5))
plot_decision_regions(log_reg.predict, X, y, title='logistic regression')
plt.show()
```

```
logistic regression, training accuracy: 0.482
```



An accuracy of about 50% is no better than guessing. We need a more flexible model.

4 The network

We use a **two-layer** network: $n^{[0]} = 2$ inputs, one hidden layer with $n^{[1]}$ units and activation function g, and one output unit with the sigmoid σ. (The input layer is not counted.) As in the lecture, the m examples are the **rows** of $\mathbf{X}$, and forward propagation for the whole batch reads

$$\mathbf{Z}^{[1]} = \mathbf{X}(\mathbf{W}^{[1]})^\top + \mathbf{b}^{[1]}, \qquad \mathbf{A}^{[1]} = g(\mathbf{Z}^{[1]}), \quad \text{shape } (m, n^{[1]}),$$
$$\mathbf{Z}^{[2]} = \mathbf{A}^{[1]}(\mathbf{W}^{[2]})^\top + \mathbf{b}^{[2]}, \quad \mathbf{A}^{[2]} = \sigma(\mathbf{Z}^{[2]}), \quad \text{shape } (m, 1).$$

Row i of $\mathbf{A}^{[2]}$ is the prediction $\hat{y}^{(i)} = P(y = 1 \mid \mathbf{x}^{(i)})$. The weight matrix of layer l has one row per unit, $\mathbf{W}^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$, exactly as `nn.Linear` stores it in PyTorch (third tutorial of this lecture).

4.1 Activation functions

We store each activation function together with its derivative, because backpropagation needs both:

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

ACTIVATIONS = {                          # name: (g, g')
    'sigmoid':  (sigmoid,                        lambda z: sigmoid(z) * (1 -
    ↪  sigmoid(z))),
```

```
    'tanh':     (np.tanh,                       lambda z: 1 - np.tanh(z) ** 2),
    'relu':     (lambda z: np.maximum(0, z),    lambda z: (z > 0).astype(float)),
    'identity': (lambda z: z,                   lambda z: np.ones_like(z)),
}
```

For the flower we use tanh in the hidden layer.

4.2 Initialisation

The weights start with small **random** values and the biases with zeros. If all weights of the hidden layer started equal, every hidden unit would compute the same output and receive the same gradient, and the units would stay identical forever (Challenge 1). Dividing by $\sqrt{n^{[l-1]}}$ keeps the values of $\mathbf{Z}^{[l]}$ of moderate size, whatever the number of inputs.

```
def initialize_parameters(n0, n1, seed=0):
    """Random weights and zero biases for a network n0 -> n1 -> 1."""
    rng = np.random.default_rng(seed)
    return {'W1': rng.normal(0, 1, (n1, n0)) / np.sqrt(n0), 'b1': np.zeros(n1),
            'W2': rng.normal(0, 1, (1, n1)) / np.sqrt(n1),  'b2': np.zeros(1)}

params = initialize_parameters(n0=2, n1=5)
for name, value in params.items():
    print(f"{name}: shape {value.shape}")
print(f"number of parameters: {sum(v.size for v in params.values())}")
```

```
W1: shape (5, 2)
b1: shape (5,)
W2: shape (1, 5)
b2: shape (1,)
number of parameters: 21
```

Layer l has $n^{[l]}(n^{[l-1]}+1)$ parameters: $5 \cdot 3 = 15$ in the hidden layer and $1 \cdot 6 = 6$ in the output layer.

4.3 Forward propagation

The function returns the predictions and a **cache** with the intermediate results, which back-propagation reuses.

```
def forward(X, params, activation='tanh'):
    """Forward propagation for a batch X of shape (m, n0). Returns A2 (m, 1) and
    ↪ a cache."""
    g, _ = ACTIVATIONS[activation]
    Z1 = X @ params['W1'].T + params['b1']       # (m, n1)
    A1 = g(Z1)                                  # (m, n1)
    Z2 = A1 @ params['W2'].T + params['b2']     # (m, 1)
    A2 = sigmoid(Z2)                            # (m, 1)
    return A2, {'Z1': Z1, 'A1': A1, 'Z2': Z2, 'A2': A2}

A2, cache = forward(X[:4], params)
for name, value in cache.items():
    print(f"{name}: shape {value.shape}")
```

```
Z1: shape (4, 5)
A1: shape (4, 5)
Z2: shape (4, 1)
A2: shape (4, 1)
```

For 4 examples, every intermediate result has one row per example.

4.4 The cost

As for logistic regression (Lecture 3a), the cost is the binary cross-entropy

$$J = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)}\log a^{[2](i)} + (1-y^{(i)})\log\left(1-a^{[2](i)}\right)\Big].$$

We compute it from $\mathbf{Z}^{[2]}$ with `np.logaddexp`, which avoids $\log(0)$, as in the logistic-regression tutorials.

```python
def compute_cost(Z2, Y):
    """Binary cross-entropy from the output scores Z2 and labels Y, both of shape
    ↪ (m, 1)."""
    return np.mean(Y * np.logaddexp(0, -Z2) + (1 - Y) * np.logaddexp(0, Z2))

Y = y.reshape(-1, 1)                # labels as a column, shape (m, 1), like A2
↪ and Z2
_, cache = forward(X, params)
print(f"cost of the untrained network: {compute_cost(cache['Z2'], Y):.4f}")
```

```
cost of the untrained network: 0.7188
```

> **Keep the shapes consistent**
>
> The predictions have shape `(m, 1)`. If the labels had shape `(m,)`, then `Y * Z2` would silently broadcast to an `(m, m)` matrix and give a wrong cost without any error message. This is why we reshape the labels to a column `Y` of shape `(m, 1)`.

5 Backpropagation

Backpropagation applies the chain rule from the output back to every parameter. For a batch, with $\odot$ the element-wise product, the lecture's equations are

$$d\mathbf{Z}^{[2]} = \mathbf{A}^{[2]} - \mathbf{Y}, \qquad \frac{\partial J}{\partial \mathbf{W}^{[2]}} = \frac{1}{m}(d\mathbf{Z}^{[2]})^{\top}\mathbf{A}^{[1]}, \qquad \frac{\partial J}{\partial \mathbf{b}^{[2]}} = \frac{1}{m}\sum_{i}(\text{row } i \text{ of } d\mathbf{Z}^{[2]}),$$

$$d\mathbf{A}^{[1]} = d\mathbf{Z}^{[2]}\,\mathbf{W}^{[2]}, \qquad d\mathbf{Z}^{[1]} = d\mathbf{A}^{[1]} \odot g'(\mathbf{Z}^{[1]}),$$

$$\frac{\partial J}{\partial \mathbf{W}^{[1]}} = \frac{1}{m}(d\mathbf{Z}^{[1]})^{\top}\mathbf{X}, \qquad \frac{\partial J}{\partial \mathbf{b}^{[1]}} = \frac{1}{m}\sum_{i}(\text{row } i \text{ of } d\mathbf{Z}^{[1]}).$$

The first line is logistic regression, with the hidden activations $\mathbf{A}^{[1]}$ as inputs. The second line passes the error back through $\mathbf{W}^{[2]}$ and through the activation function. The third line is again "error times input", now for the hidden layer.

```python
def backward(X, Y, params, cache, activation='tanh'):
    """Gradients of the cost with respect to all parameters, as a dictionary."""
    _, g_prime = ACTIVATIONS[activation]
    m = X.shape[0]
    dZ2 = cache['A2'] - Y                          # (m, 1)
    dW2 = dZ2.T @ cache['A1'] / m                  # (1, n1)
    db2 = dZ2.sum(axis=0) / m                      # (1,)
    dA1 = dZ2 @ params['W2']                       # (m, n1)
    dZ1 = dA1 * g_prime(cache['Z1'])               # (m, n1)
    dW1 = dZ1.T @ X / m                            # (n1, n0)
    db1 = dZ1.sum(axis=0) / m                      # (n1,)
    return {'W1': dW1, 'b1': db1, 'W2': dW2, 'b2': db2}
```

Each gradient has the same shape as its parameter, so the update $\theta \leftarrow \theta - \alpha\, \partial J/\partial\theta$ is a single subtraction.

5.1 Check 1: the lecture's worked example

In Part 7c of the lecture, a $2 \to 2 \to 1$ network with sigmoid activations was computed by hand for one example $\mathbf{x} = (1, 2)$ with label $y = 1$. We reproduce it:

```python
example = {'W1': np.array([[0.1, 0.2], [-0.3, 0.4]]), 'b1': np.array([0.0, 0.1]),
           'W2': np.array([[0.5, -0.6]]),             'b2': np.array([0.2])}
x_ex, y_ex = np.array([[1.0, 2.0]]), np.array([[1.0]])

a2, cache_ex = forward(x_ex, example, activation='sigmoid')
print(f"z1 = {cache_ex['Z1'][0]}, a1 = {cache_ex['A1'][0]}")
print(f"z2 = {cache_ex['Z2'][0, 0]:.4f}, a2 = {a2[0, 0]:.4f}, L =
 ↪  {compute_cost(cache_ex['Z2'], y_ex):.4f}")

grads_ex = backward(x_ex, y_ex, example, cache_ex, activation='sigmoid')
for name, value in grads_ex.items():
    print(f"dJ/d{name} =\n{value}")
```

```
z1 = [0.5 0.6], a1 = [0.6225 0.6457]
z2 = 0.1238, a2 = 0.5309, L = 0.6331
dJ/dW1 =
[[-0.0551 -0.1102]
 [ 0.0644  0.1288]]
dJ/db1 =
[-0.0551  0.0644]
dJ/dW2 =
[[-0.292  -0.3029]]
dJ/db2 =
[-0.4691]
```

All values agree with the tables on the slides "Forward pass with numbers" and "Backward pass with numbers".

5.2 Check 2: finite differences

As in the linear-regression tutorials, we compare every gradient entry with a central finite difference, here for random parameters and data:

```python
def numerical_gradients(X, Y, params, activation, eps=1e-6):
    """Central-difference approximation of all gradients, entry by entry."""
    grads = {}
    for name, value in params.items():
        grads[name] = np.zeros_like(value)
        for idx in np.ndindex(value.shape):          # every entry of the array
            original = value[idx]
            value[idx] = original + eps
            J_plus = compute_cost(forward(X, params, activation)[1]['Z2'], Y)
            value[idx] = original - eps
            J_minus = compute_cost(forward(X, params, activation)[1]['Z2'], Y)
            value[idx] = original                    # restore the parameter
            grads[name][idx] = (J_plus - J_minus) / (2 * eps)
    return grads

rng = np.random.default_rng(1)
X_chk, Y_chk = rng.normal(size=(8, 2)), rng.integers(0, 2, size=(8, 1))
params_chk = initialize_parameters(2, 3, seed=1)
grads = backward(X_chk, Y_chk, params_chk, forward(X_chk, params_chk)[1])
grads_fd = numerical_gradients(X_chk, Y_chk, params_chk, 'tanh')
for name in params_chk:
    print(f"{name}: largest difference {np.max(np.abs(grads[name] -
    ↪ grads_fd[name])):.1e}")
```

```
W1: largest difference 3.1e-11
b1: largest difference 5.5e-11
W2: largest difference 6.8e-11
b2: largest difference 9.4e-12
```

The differences are at the level of rounding errors, so `backward` is correct.

6 Training with gradient descent

Training repeats four steps: forward pass, cost, backward pass, and the update $\theta \leftarrow \theta - \alpha\, \partial J/\partial \theta$ for every parameter.

```python
def train(X, Y, n_hidden, activation='tanh', alpha=1.2, n_iters=5000, seed=0):
    """Batch gradient descent for a network n0 -> n_hidden -> 1."""
    params = initialize_parameters(X.shape[1], n_hidden, seed)
    J_history = np.zeros(n_iters)
    for t in range(n_iters):
        _, cache = forward(X, params, activation)
        J_history[t] = compute_cost(cache['Z2'], Y)
        grads = backward(X, Y, params, cache, activation)
        for name in params:
            params[name] -= alpha * grads[name]
    return params, J_history

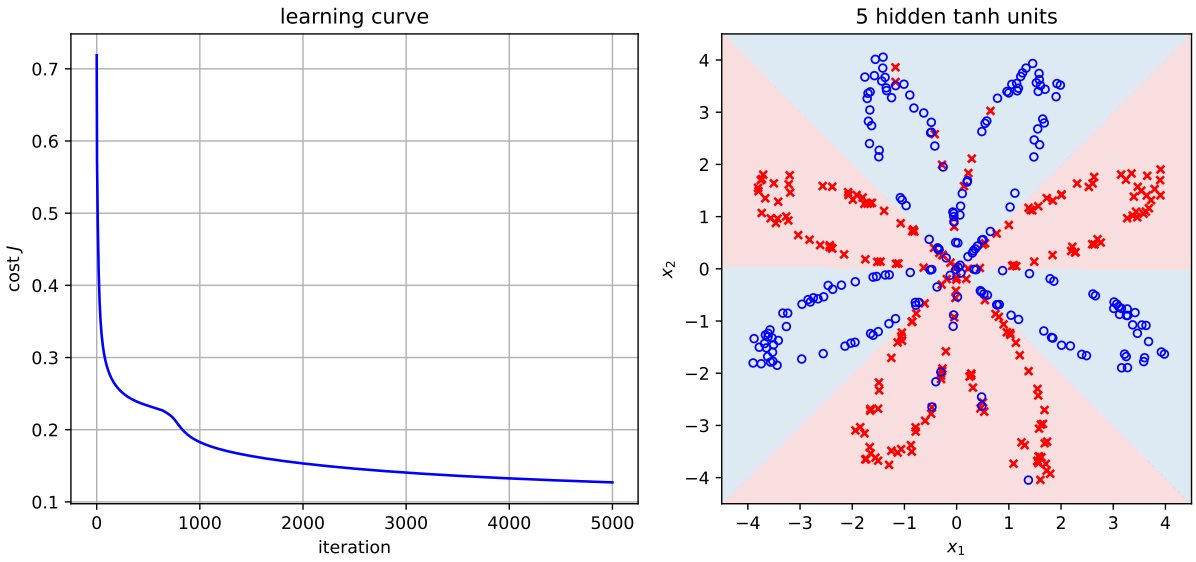
def predict(X, params, activation='tanh'):
    """Class predictions (0 or 1) with the threshold 0.5."""
    A2, _ = forward(X, params, activation)
    return (A2[:, 0] >= 0.5).astype(int)

params, J_hist = train(X, Y, n_hidden=5)
```

```
print(f"training accuracy: {np.mean(predict(X, params) == y):.3f}")

fig, ax = plt.subplots(1, 2, figsize=(10, 4.5), constrained_layout=True)
ax[0].plot(J_hist, 'b-')
ax[0].set(xlabel='iteration', ylabel='cost $J$', title='learning curve')
ax[0].grid(True)
plot_decision_regions(lambda Xg: predict(Xg, params), X, y, ax=ax[1], title='5
↪ hidden tanh units')
plt.show()
```

```
training accuracy: 0.943
```

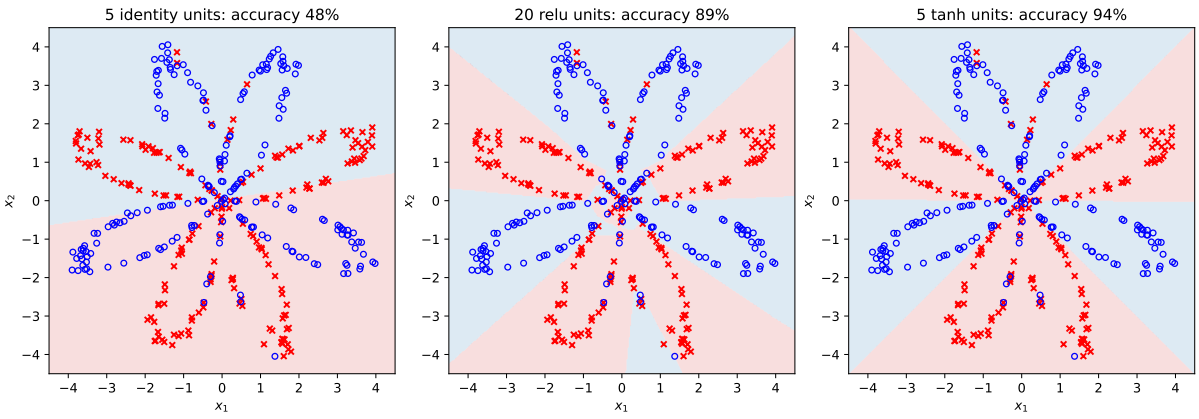


With only five hidden units, the network classifies about 94% of the points correctly. Its decision boundary divides the plane into wedges that follow the petals, whereas logistic regression could only draw a single straight line.

7 Why must the hidden activation be nonlinear?

We train the same network with three different hidden activations. With the identity, $g(z) = z$, the two layers collapse into a single linear map, $\mathbf{a}^{[2]} = \sigma(\mathbf{W}'\mathbf{x} + \mathbf{b}')$, so the network is logistic regression again.

```
fig, ax = plt.subplots(1, 3, figsize=(13, 4.5), constrained_layout=True)
for a, (activation, n_hidden, alpha) in zip(ax, [('identity', 5, 0.1), ('relu',
↪ 20, 0.5), ('tanh', 5, 1.2)]):
    p, _ = train(X, Y, n_hidden, activation, alpha)
    acc = np.mean(predict(X, p, activation) == y)
    plot_decision_regions(lambda Xg: predict(Xg, p, activation), X, y, ax=a,
                          title=f'{n_hidden} {activation} units: accuracy
↪ {acc:.0%}')
plt.show()
```



- **Identity**: a single straight boundary and the accuracy of logistic regression, however many hidden units we use.
- **ReLU**: a boundary made of straight pieces; with 20 units it reaches about 88%.
- **tanh**: five units are enough to cut the plane into wedges that match the petals.

8 Challenges

1. Initialise all weights to 0 instead of random values and train again. Print `params['W1']` after training. What do you observe, and why?
2. Train with $n^{[1]} = 1, 2, 3, 5, 20$ and 50 hidden units. How do the accuracy and the decision boundary change? Would you trust the model with 50 units on new points?
3. Train with learning rates $\alpha = 0.01$, 0.1 and 5, and compare the learning curves.
4. A network has layers of sizes $10 \to 8 \to 8 \to 3$. What are the shapes of $\mathbf{W}^{[1]}$, $\mathbf{W}^{[2]}$ and $\mathbf{W}^{[3]}$, and how many parameters does it have?

9 Summary

- Forward propagation for a batch (rows = examples): $\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}(\mathbf{W}^{[l]})^\top + \mathbf{b}^{[l]}$, $\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$.
- Backpropagation starts from $d\mathbf{Z}^{[2]} = \mathbf{A}^{[2]} - \mathbf{Y}$ and works backwards: weight gradients are $\frac{1}{m}(d\mathbf{Z})^\top \mathbf{A}_{\text{prev}}$, and the error passes back as $d\mathbf{A}_{\text{prev}} = d\mathbf{Z}\,\mathbf{W}$, then through g'.
- Always check gradients, on an example with known values and with finite differences.
- Initialise the weights randomly to break the symmetry between units.
- Without a nonlinear activation, a network is no more powerful than logistic regression.

Next: networks with any number of layers.